Algorithmization of Objects in EO Programs

Anonymous Author(s)

Abstract

To be written later...

CCS Concepts: • **Object-Oriented Programming**;

Keywords: Object-Oriented Programming, Devirtualization

1 Introduction

On one hand, there are pure object-oriented programming languages, such as Ruby or Self [8], which obey the principle "everything is an object" [9, p.121]. In such languages even primitive data are objects, such as integers or Boolean values. On the other hand, there are also "hybrid" programming languages, such as C++ or Java. They distinguish objects and primitive data types such as numbers or arrays. Obviously, hybrid languages demonstrate higher performance due to elimination of object allocation [4] and dynamic dispatch overhead [5, 6], especially in data-intensive algorithms [1, 3].

There is an opportunity to boost performance of programs written in a pure object-oriented language, if the language has a Foreign Function Interface (FFI) to a hybrid or an imperative language. For example, before execution, parts of a Ruby program that most actively manipulate with data, may automatically be re-written in C and then compiled into binaries. At runtime, the binaries will be called from Ruby via FFI. They will receive data from Ruby, process it, and communicate back to Ruby by a) returning new data and b) continuously exchanging data via some protocol. Such an optimization may be suitable mostly for interpreted languages.

One of the challenges of such an optimization is related to the effectiveness of the selection of a code snippet in a object-oriented program: it is expected that the performance of the program will be increased after the translation of the snippet to a different language. We assume that object manipulations, such as allocation and dynamic dispatch, are done faster by the original object-oriented language, while arithmetical operations are faster in the hybrid language. Thus, selecting a snippet of the code that is too big, will lead to performance losses because of slower objects manipulations in the hybrid language. At the same time, a very small snippet will either give no performance gain or even cause performance losses due to the necessity to transfer data between languages. Thus, in order to enable the optimization, it is mandatory to have an algorithm that, taking a program as an input and performance properties of both languages, suggest the most suitable snippet for translation.

In this paper we suggest such an algorithm. This algorithm founds all code snippets, the translation of which into another programming language will reduce the expected execution time of the program. In addition, the algorithm filters out those code snippets that cannot be translated into another programming language. After that, the algorithm allocates a set of code snippets that can be translated simultaneously and the translation of which will minimize the execution time of the program. The algorithm reduces the choice of code snippet for translation to the maximum clique problem.explain how it works, in exactly one short paragraph: FIXED

```
class F
  def eof; ...; end # TRUE if end of file
  def next; ...; end # Reads next line
end
f = F.new
a = 0
loop do
  t = f.next.to_i
  next if t % 3 != 0
  a += t * t
  break if f.eof
end
puts "a = #{a}"
```

Figure 1. Simple Ruby program. This program contains a fairly large number of mathematical operations, and its example will show optimization using code snippet translation. write up the caption: FIXED

In order to test the effectiveness of the algorithm we used EO, a pure object-oriented programming language, as the original language, and Rust as a lower-level language. We applied our method to a few EO programs and calculated their performance before the application and after. Our results demonstrate that the algorithmization increased their performance by give specifics!. The method may be applied to other programming languages.

The contribution we make is two fold: 1) the method itself and 2) the software tool for algorithmization of EO programs.

The rest of the paper has the following structure: Section 2 presents a motivating example to demonstrate performance effect of algorithmization. Section 4 introduces the method informally. Section 5 formally describes the method. Section 9 demonstrates we applied the method to a few programs and what results achieved. Section 10 provides an overview of our results and sheds light on future research. Lastly, Section 11 summarizes the findings of this study.

```
loop do
  t = f.next.to_i
  next if t % 3 != 0
  a = ffi("return x + y * y;")
    .with("x", a)
    .with("y", t)
    .exec()
  break if f.eof
end
```

Figure 2. Ruby program after algorithmization. We chose some code snippet and converted it to C. After this conversion, the program execution time decreased by . write up the caption: FIXED, but update

2 Motivating Example

A Ruby program in Fig. 1 iteratively reads lines from a text file, where each line presumably contains a textual form of an integer. If the result of the division of the integer by three is not equal to zero, the algorithm moves to the next line in the file. Otherwise, the number is multiplied by itself and then added to the accumulator. At the end of the file the algorithm stops and prints the value of the accumulator.

Figure 2 demonstrates the simplest algorithmization of the code. Everything outside of the `loop` is skipped for the sake of brevity. The builder function `ffi` takes a string with a simple C function. It must be compiled to a binary code before Ruby script is executed (this feature must be embedded into Ruby interpreter). At runtime, `x` and `y` are replaced with the values of Ruby variables `a` and `b` and then the C function is evaluated. This optimization moves two arithmetic operations to a lower-level language: multiplication and addition. They definitely are faster in C, taking into account that in Ruby they are not operators but methods of class `Integer`. However, the cost of calling `ffi` may be larger than the effect of optimization.

Figure 3 demonstrates a more complex algorithmization with a wider scope, which may look like giving a bigger effect. Here, the entire `loop` cycle is inside the C function. A reverse access to Ruby objects is implemented in `f` parameter passed into it. The cost of such a reverse call to Ruby may be higher than the effect of optimization, which is gained due to `atoi` C function instead or Ruby method `String.to_i`. Keeping a positive balance between effect and cost is what is expected from algorithmization algorithm.

3 Related Work

The most similar algorithm to the one being developed is the packaging and unpacking mechanism implemented in Java. In Java, there are special wrapper classes for primitive data types such as *int* or *double*. It is possible to convert a primitive data type into a wrapper class object and vice

```
a = ffi("""
  #include <stdlib.h>
  int t;
  int a = 0;
  while (true) {
    t = atoi(f.next());
    if (t % 3 != 0) {
      next;
    }
    a += t * t;
    if (f.eof()) {
      break;
    }
  }
  return a;
""").with("f", f).eval()
```

Figure 3. Ruby program after the algorithmization with a bigger snippet write up the caption

(these processes are called packing and unpacking). In some cases, Java can optimize the code, convert wrapper classes into primitive data types and perform calculations in them to reduce the expected execution time of the program (the process is called autoboxing). Despite the similarity with the developed algorithm at first glance, this mechanism is very different from the developed algorithm. With autoboxing, data is simply transformed, while fragments of the program code are not replaced with fragments of code in another language and the source code of the program is almost not affected. The performance gain thus obtained during the operation of the program remains quite small.

Let's find out what similar has been done. I believe, we should mention JIT and AOT technologies. We should also mention existing analysis of OOP performance – why we think OOP is slow? Let's also say something about existing studies of FFI performance. FIXED

4 Overview

Here, let's explain how the method works in simple terms, without too much details.

5 Method

5.1 Graph definitions

In this subsection, it is described definitions in graph theory, which will be needed for algorithmization task.

Definition 5.1 (Graph). A *graph* $G \in \mathbb{G}$ is a pair $\langle V_G, E_G \rangle$, where V_G is a set whose elements are called *vertices* and belonging to the set $\mathbb{V}$, and E_G is a set of ordered pairs of vertices, belonging to the set $\mathbb{V}^2$. Vertices are comparable. Elements of E_G are called *edges*.

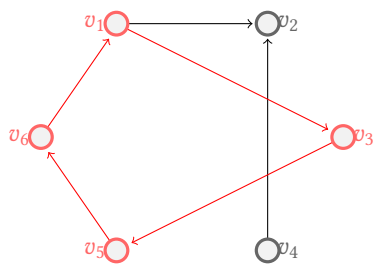

Figure 4. Consider $V_G = \{v_1, v_2, v_3, v_4, v_5, v_6\}$, $E_G = \{\langle v_1, v_2\rangle, \langle v_1, v_3\rangle, \langle v_3, v_5\rangle, \langle v_4, v_2\rangle, \langle v_5, v_6\rangle, \langle v_6, v_1\rangle\}$. Then the sequence v_1, v_3, v_5, v_6, v_1 is a cycle (the vertices and edges included in the specified cycle are indicated in red in the figure).

Definition 5.2 (Cycle)**.** For any given graph G, a *cycle* is a sequence $v_0, v_1, \ldots, v_n$ where the following are both true:

- $\forall i \in [0; n] : v_i \in V_G$, and $v_0 = v_n$, and $n \geq 2$;
- $\forall i \in [0; n-1] : \langle v_i, v_{i+1}\rangle \in E_G$.

Example 5.3. Consider $V_G = \{v_1, v_2, v_3, v_4, v_5, v_6\}$, $E_G = \{\langle v_1, v_2\rangle, \langle v_1, v_3\rangle, \langle v_3, v_5\rangle, \langle v_4, v_2\rangle, \langle v_5, v_6\rangle, \langle v_6, v_1\rangle\}$. Then, the sequence v_1, v_3, v_5, v_6, v_1 is a cycle. Figure 4 demonstrates this graph visually.

Definition 5.4 (Members of Cycles)**.** For any given graph G, a $\mathbf{Q}(G) = \{v|\ (v \in V_G) \wedge (k \geq 1) \wedge (\exists v_1, \ldots, v_k \in V_G :$ sequence $v, v_1, \ldots, v_k$ is cycle$)\}$ is a function, evaluates all vertices, which are included in at least one cycle. "a function that evaluates a set of vertices" - it's not clear exactly what is the result of this evaluation? what does this function do?

Definition 5.5 (Simple Path)**.** For any given graph G, a sequence $v_0, v_1, \ldots, v_n$ is a *simple path* if all of the following conditions are met:

- $n \geq 1$;
- $\forall i \in [0; n] :\ v_i \in V_G$;
- $\forall i \in [0; n-1] :\ \langle v_i, v_{i+1}\rangle \in E_G$;
- all v are unique.

Definition 5.6 (Reachability Predicate)**.** For any given graph G, a predicate $\mathbf{P}(G, v_0, v_n)$ holds, if one of the conditions is met

- $v_0 = v_n$;
- $\exists v_1, v_2, \ldots, v_{n-1} \in V_G$ such that the sequence $v_0, v_1, \ldots, v_{n-1}$ is a *simple path* and $\langle v_{n-1}, v_n\rangle \in E_G$.

Definition 5.7 (Children)**.** For any given vertex v in a graph G, the function $\mathbf{C}(G, v)$ returns all *children* of the vertex v, such that $\mathbf{C}(G, v) = \{v_1 | v_1 \in V_G \wedge \langle v, v_1\rangle \in E_G\}$.

Definition 5.8 (Source)**.** In any given graph G, a vertex $v_0 \in V_G$ is a *source* of the graph if $\nexists\, v_1 \in V_G : \langle v_1, v_0\rangle \in E_G$.

Definition 5.9 (Sink)**.** In any given graph G, a vertex $v \in V_G$ is a *sink* of the graph if $\nexists\, v_1 \in V_G : \langle v_0, v_1\rangle \in E_G$.

Definition 5.10 (Ancestors)**.** For any given vertices v_0, v_1 in any given graph G, a $\mathbf{A}(G, v_0, v_1) = \{v_2 | v_2 \in V_G \wedge \mathbf{P}(G, v_0, v_2) \wedge \mathbf{P}(G, v_2, v_1)\}$ is function, which finds all vertices, which lie at least on one path from v_0 to v_1.

Definition 5.11 (Descendants)**.** For any given vertex v_0 in any given graph G, function $\mathbf{D}(G, v_0) = \{v_1 | v_1 \in V_G \wedge \mathbf{P}(G, v_0, v_1)\}$ finds all vertices, reachable from vertex v_0.

Definition 5.12 (Back-edge)**.** For any given graph G, edge $e = \langle v_1, v_2\rangle \in E_G$ is *back-edge* if $\exists\, v_3 \in \mathbf{D}(G, v_2) : \langle v_3, v_1\rangle \in E_G$.

5.2 Control-Flow Graph

In this subsection, mathematical descriptions of the objects necessary for setting the algorithmization problem will be given. In Section 5.3 the mathematical objects described here and the real entities found in the program code will be matched.

Definition 5.13 (Programming Objects)**.** A *programming object (PO)* $p \in \mathbb{P}$ is a pair $\langle \xi_p, \omega_p\rangle$, where ξ is number and ω is number.

Definition 5.14. *Control-flow graph* $C \in \mathbb{T}$ is a tuple $\langle G_C, O_C, \mathrm{Y}_C\rangle$, where G_C is graph (which has only one source), $O_C \subseteq \mathbb{P}$ and Y_C is a function, which maps vertex and its color (Definition 5.16). Why do we need to put the source in the definition of the CFG, if the source is computable from G? also, it's unclear what is "map" - maybe you meant "function"?

Definition 5.15 (Root of Control-Flow Graph)**.** For any given control-flow graph C, a vertex ρ_c is a *root of control-flow graph* C. According to Definition 5.14, G_C has only one source. ρ_c is a previously mentioned source of graph G_C.

Definition 5.16 (Color of Vertex)**.** For any given vertex v in any given control-flow graph C, a function $\mathrm{Y}(C, v)$ evaluates color of vertex v. $\forall C' \in \mathbb{T} :\ \forall v' \in V_{G_C} :\ \mathrm{Y}(C', v') \in \mathbb{L}$ ($\mathbb{L}$ is described in Definition 5.17). There are restrictions on the Y function, described in Lemma 5.18 and Lemma 5.19. This topic is discussed in more details at (Example 5.29). again, why it's a "map" instead of a "function "?

Definition 5.17 (Set of Colors)**.** A set $\mathbb{L} = \{\mathcal{A}, \mathcal{B}\}$ contains all possible values of the Y returned value for each of the vertices of any control-flow graph. $\mathcal{A}$ and $\mathcal{B}$ are abstract constants. This topic is discussed in more details at (Example 5.33).

Lemma 5.18 (Restriction on Color of Root)**.** *For any source v in any given control-flow graph C, $\mathrm{Y}(C, v) = \mathcal{A}$.*

Lemma 5.19 (Restriction on Color of Sink)**.** *For any sink v in any given control-flow graph C, $\mathrm{Y}(C, v) = \mathcal{A}$.*

Definition 5.20 (Laboriousness)**.** For any given vertex v in any given control-flow graph C and for any color $k \in \mathbb{L}$, a function $\mathbf{M}(C, v, k)$ evaluates *laboriousness*. $\forall C \in \mathbb{T} :\ \forall v \in$

V_{G_C} : $\forall k \in \mathbb{L}$: $\mathbf{M}(C, v, k) \in \mathbf{M}(C, v, k) \geq 0$. This topic is discussed in more details at (Example 5.30). Don't say "read more.." - this is not acceptable in academic writing. Instead, say "This topic is discussed in more details at..." However, I don't recommend to do this in a "Definition", since it's very informal. A "definition" is like a law, it may not have double meaning.

Definition 5.21. For any given vertex v in any given control-flow graph C, a function $\mathbf{R}(C, v)$ evaluates *set of readable programming objects*, which is a subset of O_C. This topic is discussed in more details at (Example 5.31). Here you call them "objects" but earlier you called them "programming objects". Why changing the name?

Definition 5.22. For any given vertex v in any given control-flow graph C, a function $\mathbf{W}(C, v)$ evaluates *set of writable programming objects*, which is a subset of O_C. This topic is discussed in more details at (Example 5.31).

Definition 5.23 (Component)**.** For any given vertex v in any given control-flow graph C and $G = G_C$, $\mathbf{J}(C, v) = \{v_1 \in V_G |\ \mathbf{TMP}(C, v, v_1)\}$, where predicate $\mathbf{TMP}(C, v, v_1)$ holds on when all of the following conditions are met:

- $\mathbf{Y}(C, v) = \mathbf{Y}(C, v_1)$;
- $\exists\, v_2, \ldots, v_k \in V_G$:
  - $\mathbf{Y}(C, v) = \mathbf{Y}(C, v_2) \wedge \cdots \wedge \mathbf{Y}(C, v) = \mathbf{Y}(C, v_k)$;
  - $\langle v_2, v_3 \rangle \in E_G \wedge \cdots \wedge \langle v_{k-1}, v_k \rangle \in E_G$;
  - $\langle v, v_2 \rangle \in E_G$;
  - $\langle v_k, v_1 \rangle \in E_G$.

In other words, $\mathbf{J}(C, v)$ is a set of vertices of graph G_C, which are reachable from vertex v by at least one path, which has all vertices with color equal with color of vertex v.

Definition 5.24 (Component estimator)**.** For any given vertex v in any given control-flow graph C and $\mathbf{Y}(C, v) = \mathcal{B}$, a function $\mathbf{O}(C, v)$ evaluates *component estimator* of vertex v. $\mathbf{O}(C, v) = v_0$ if all of the following conditions are met:

- $v \in \mathbf{J}(C, v_0)$;
- $\nexists v_1 \in V_G$: $\mathbf{Y}(C, v_1) = \mathcal{B} \wedge v_0 \in \mathbf{J}(C, v_1)$.

In other words, $\mathbf{O}(C, v) = v_0$ is a vertex, for which $\mathbf{Y}(C, v_0) = \mathcal{B}$ (according to Definition 5.23) and component $\mathbf{J}(C, v_0)$ is the largest of possible components, which contains vertex v.

Definition 5.25. For any given control-flow graph C, $\mathbf{X}(C) = \{c' \in \mathbb{T} :\ G_c = G_{c'} \wedge O_C = O_{C'} \wedge \rho_c = \rho_{c'}\}$.

In other words, $\mathbf{X}(C)$ is set of control-flow graphs, which has same structure with C, but differ only by $\mathbf{Y}$. In next sections, it will be shown, that algorithm, which is described in this article, will find "the optimal" $\mathbf{Y}$ for specified control-flow graph.

5.3 Relationship CFG with real programs

As mentioned earlier, CFG C is $\langle G_C, O_C, \rho_c, \mathbf{Y}_C \rangle$. But also control-flow graph is a representation of all possible ways to execute a program in the form of a graph. For any program in any programming language, it can be built control flow graph. In this subsection, we will look at all the definitions described in Section 5.2, and match the mathematical objects introduced there with the real ones.

```
int f = 1;
for (int i = 1; i <= 6; ++i) {
    f = f * i;
}
```

Figure 5. Simple fragment of c++ code.

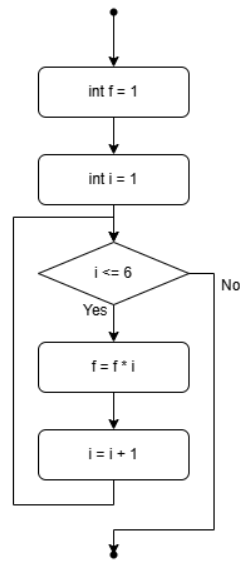

Figure 6. The representations of code fragments, matched with vertices of G of control-flow graph for program from Fig. 5

Example 5.26. This example demonstrates a simple C++ program which counts factorial (Fig. 5) and at representations of control-flow graph for this program (Fig. 6 and Fig. 7).

The source program is divided into blocks of program commands. Fig. 6 shows the splitting of the program described in Fig. 5 into blocks. Each such block has its own vertex in G_C.

Example 5.27. Programming object (PO) (Definition 5.13) is a representation of object used in the program. This programming object can be a variable of a primitive data type, an object variable, as well as an array or any other container. There are 2 programming objects in this program: variable f - $\langle 1,\ 4 \rangle$ and variable i - $\langle 2,\ 4 \rangle$. First element of pair is unique ID and second is weight of programming object in bytes. Next, for simplicity, when it comes to an programming object, we will refer to its ID.

Example 5.28. $\mathbb{L}$ (Definition 5.20) is set of programming languages where code execution is available to us. In our

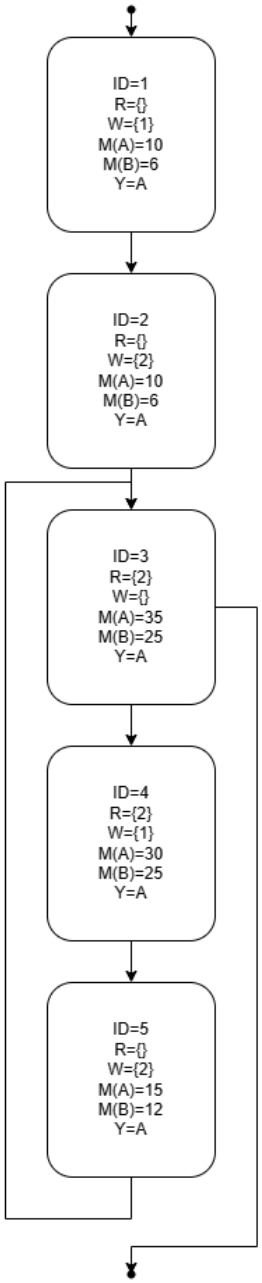

Figure 7. The control flow graph for program from Fig. 5. For convenience, each vertex has been assigned an ID to make it easier to distinguish them from each other.

case, when there is some programming language that supports FFI for another programming language, the original programming language is designated $\mathcal{A}$ (name it origin), and the FFI language is $\mathcal{B}$ (name it FFI language).

Example 5.29. All of the vertices have color - Y (Definition 5.16). Color of vertex v means, that all programming commands, corresponding to vertex v must be executed on programming language $\mathbf{Y}(C, v)$. As example, vertex u with ID = 2 has color $\mathcal{A}$, so all commands corresponding to vertex u must be executed on the origin programming language.

Example 5.30. $\mathbf{M}(C, v, l)$ (Definition 5.20) is a function, which evaluates expected temporary labor costs of executing programming commands, corresponding to vertex v on programming language l. The execution time of the code corresponding to vertex v is measured in conventional units of labor. As example, vertex u with ID = 2 can be executed on origin language in 10 conventional units, while it can be executed on FFI language in 6 conventional units.

Example 5.31. All of the vertices has **R** and **W** sets Definition 5.21 and Definition 5.22. $\mathbf{R}(v)$ is the set of programming objects, which are used in any of commands, corresponding to vertex v and are not modified by these commands. $\mathbf{W}(v)$ is the set of programming objects, which are modified by commands, corresponding to vertex v. As example, command f̲ = f * i;, corresponding to vertex u with ID= 4, modifies the value of variable f (it has ID = 1), so $\mathbf{W}(u) = \{1\}$. Also, this code fragment uses value of variable i (it has ID = 2), but do not modify it, so $\mathbf{R}(u) = \{2\}$.

Example 5.32. For $C \in \mathbb{T}$, ρ_C is entry point of program. If the code shown in Fig. 5 were a complete program, then vertex u with ID= 1 would be the entry point to this program.

For any given control-flow graph C, $\mathbf{X}(C)$ is set of all possible control-flow graphs, which have same G_C, O_C and ρ_c, but differing only in $\mathbf{Y}_C$. In other words, it is the same control-flow graph, but in which the colors of the vertices differ.

Example 5.33. Each $C \in \mathbb{T}$ can be associated with a program written in language $\mathcal{A}$, in which all code fragments corresponding to the vertices of G_C colored $\mathcal{B}$ are replaced by code insertions in language $\mathcal{B}$ with using FFI.

The purpose of the algorithm described in this article is to select for specified CFG C from the $\mathbf{X}(C)$ one control-flow graph, which is the most optimal by some criterion. A description of this criterion is given in the following sections.

5.4 Data transferring

If a code fragment executed in the programming language $\mathcal{A}$ is followed by a code fragment executed in the programming language $\mathcal{B}$, then it is necessary to transfer data to this code fragment, as well as pick up the data returned by this code fragment. It takes some kind of laboriousness to transfer and return this data. So, the data-transferring function calculates this laboriousness. Data transferring function assumes that the laboriousness of transporting data are proportional to the volume of data being transported. It is argued that the laboriousness of transporting data can be described by a function of the form $\mathbf{M} = \alpha + \beta x$, where x is the total amount of memory occupied by the transported data, and $\alpha \in \mathbb{R}$ and

$\beta \in \mathbb{R}$ are specified coefficients. In this subsection we will describe this function.

Definition 5.34 (Imported Objects)**.** For any given edge $e = \langle v_0, v_1 \rangle$ in any given control-flow graph C, such that $\mathbf{Y}(C, v_0) = \mathcal{A}$ and $\mathbf{Y}(C, v_1) = \mathcal{B}$, define the function $\mathbf{I}(C, v_0, v_1)$, which evaluates set of imported programming objects:

$$\phi_1 = \bigcup_{v_2 \in \mathbf{A}_{G_C, \rho_c}(v_0)} (\mathbf{R}(C, v_2) \cup \mathbf{W}(C, v_2))$$
$$\phi_2 = \bigcup_{v_2 \in \mathbf{J}(C, v_1)} (\mathbf{R}(C, v_2) \cup \mathbf{W}(C, v_2))$$
$$\mathbf{I}(C, v_0, v_1) = \phi_1 \cap \phi_2$$

$\mathbf{I}(C, v_0, v_1)$ is the set of programming objects, which were presented in $\mathbf{R}$ or $\mathbf{W}$ of any vertex on any path from ρ_c to v_0 and are presented in $\mathbf{R}$ or $\mathbf{W}$ of any vertex from $\mathbf{D}(G, v_1)$. In other words, these are programming objects that need to be imported to code fragment that should been converted into an insertion of code in the FFI language.

Definition 5.35 (Exported Objects)**.** For any given $e = \langle v_0, v_1 \rangle$ in any given control-flow graph C, such that $\mathbf{Y}(C, v_0) = \mathcal{B}$ and $\mathbf{Y}(C, v_1) = \mathcal{A}$, define the function $\mathbf{E}(C, v_0, v_1)$, which evaluates set of exported programming objects:

$$\phi_1 = \bigcup_{v_2 \in \mathbf{J}(C, \mathbf{O}(C, v_0))} \mathbf{W}(C, v_2)$$
$$\phi_2 = \bigcup_{v_2 \in \mathbf{D}(G, v_1)} (\mathbf{R}(C, v_2) \cup \mathbf{W}(C, v_2))$$
$$\mathbf{E}(C, v_0, v_1) = \phi_1 \cap \phi_2$$

$\mathbf{E}(C, v_0, v_1)$ is the set of programming objects, which were modified in component (Definition 5.23) of vertex v_0 and are presented in $\mathbf{R}$ or $\mathbf{W}$ of any descendant of vertex v_1. In other words, these are programming objects that need to be exported from code fragment that should be converted into an insertion of code in the FFI language.

Definition 5.36 (Transporting Cost)**.** For any given edge $e = \langle v_0, v_1 \rangle$ in any given control-flow graph C, as well as parameters $\alpha \in \mathbb{R} \wedge \alpha \geq 0$ and $\beta \in \mathbb{R} \wedge \beta > 0$, define function $\mathbf{T}(C, \alpha, \beta, v_0, v_1)$, which evaluates laboriousness of transporting data from vertex v_0 to vertex v_1.

- If $\mathbf{Y}(C, v_0) = \mathbf{Y}(C, v_1)$, then $\mathbf{T}(C, \alpha, \beta, v_0, v_1) = 0$
- If $\mathbf{Y}(C, v_0) = \mathcal{A} \wedge \mathbf{Y}(C, v_1) = \mathcal{B}$, then
$$\mathbf{T}(C, \alpha, \beta, v_0, v_1) = \alpha + \beta \sum_{o \in \mathbf{I}(C, v_0, v_1)} \omega_o$$
- If $\mathbf{Y}(C, v_0) = \mathcal{B} \wedge \mathbf{Y}(C, v_1) = \mathcal{A}$, then
$$\mathbf{T}(C, \alpha, \beta, v_0, v_1) = \alpha + \beta \sum_{o \in \mathbf{E}(C, v_0, v_1)} \omega_o$$

5.5 Target Function

For any given control-flow graph C, algorithm, which will be described in this article later, will search 'the best' control-flow graph among $\mathbf{X}(C)$ by some criteria. The criteria is to minimize the function of expected time of execution of the program, corresponding to specified control-flow graph. In this subsection will be described $\mathbf{L}$-function, which evaluates expected time of execution of the program, corresponding to specified control-flow graph.

Definition 5.37 (Target Function)**.** For any given vertex v in any given control-flow graph C, any $\alpha \in \mathbb{R} \wedge \alpha \geq 0$, any $\beta \in \mathbb{R} \wedge \beta > 0$ and any $\gamma \in \mathbb{R}$, define recursively function $\mathbf{H}(G, v, \alpha, \beta)$:

- If $\mathbf{C}(G_C, v) = 0$, then $\mathbf{H}(C, v, \alpha, \beta) = \mathbf{M}(C, v, \mathbf{Y}(C, v))$;
- If $\mathbf{C}(G, v) \neq 0$, then

$$\phi_1 = \mathbf{M}(C, v, \mathbf{Y}(C, v))$$
$$\psi = \frac{1}{|\mathbf{C}(G_C, v)|}$$
$$\lambda_1 = \{v' |\ v' \in \mathbf{C}(G_C, v) \wedge \overline{v' \in \mathbf{Q}(C)}\}$$
$$\mu_1 = \sum_{v' \in \lambda_1} (\mathbf{H}(C, v') + \mathbf{T}(C, \alpha, \beta, v, v'))$$
$$\lambda_2 = \{v' |\ v' \in \mathbf{C}(G, v) \wedge (v' \in \mathbf{Q}(G_C))\}$$
$$\mu_2 = \sum_{v' \in \lambda_2} \mathbf{T}(C, \alpha, \beta, v, v')$$
$$\mathbf{H}(C, v, \alpha, \beta) = \phi_1 + \psi * (\mu_1 + \gamma \mu_2)$$

$\mathbf{H}(C, v, \alpha, \beta)$ can be interpreted as a mathematical expectation of the execution time of the program corresponding to control-flow graph C, if its execution began with the part of the code corresponding to the vertex v.

Definition 5.38 (Final Target Function)**.** For any given control-flow graph C, any $\alpha \in \mathbb{R} \wedge \alpha \geq 0$ and any $\beta \in \mathbb{R} \wedge \beta > 0$, $\mathbf{L}(C, \alpha, \beta) = \mathbf{H}(C, \rho_c, \alpha, \beta)$.

This function evaluates the expected execution time of the program corresponding to the control-flow graph passed as an argument of $\mathbf{L}$-function.

Definition 5.39 (Ideal)**.** For any given control-flow graph C, $C' \in \mathbf{X}(C)$ is *ideal* if $\nexists C_2 \in \mathbf{X}(C) : C_2 \neq C' \wedge \mathbf{L}(C_2) < \mathbf{L}(C')$.

Lemma 5.40. Ideal *is a control-flow graph that provides the minimum possible expected execution time of the program corresponding to matched control-flow graph.*

5.6 Restrictions imposed on code fragments transformed into $\mathcal{B}$

In this section, the concept of a fragment is introduced - a set of vertices of a control-flow graph. The concept of a correct fragment will be introduced - a set of vertices of a graph, all parts of the code corresponding to which can be transformed as a single code insertion with using FFI.

Definition 5.41 (Fragment)**.** For any given vertices S and F in any given graph G, $\mathbf{\Phi}(\mathrm{G, S, F}) = \{v \mid v \in V \wedge \exists v' \in V_G : (\mathbf{P}(G, \mathrm{S}, v') \wedge \mathbf{P}(G, v', \mathrm{F}))\}$. Each fragment has unique id (for fragment F denote as ξ_F).

Definition 5.42 (Entry-point and Exit-point)**.** For any given vertices S and F in any given graph G, $\mathbf{\Phi}(G, \mathrm{S}, \mathrm{F})$ denote *entry-point* = S and *exit-point* = F.

Definition 5.43 (Fragment Outputs)**.** For any given vertices S and F in any given graph G, $f = \Phi(G, \mathrm{S}, \mathrm{F})$, define a function $\zeta(G, f) = \bigcup_{v \in \Phi(G,\mathrm{S},\mathrm{F})} \{v' \mid v' \in \mathbf{C}(G, v) \wedge v' \notin \Phi(G, \mathrm{S}, \mathrm{F}) \wedge |\mathbf{C}(G, v')| \neq 0\}$, which evaluates *fragment outputs*.

For any fragment f, *fragment outputs* is a set of vertices, which are not included in f and for any vertex u from *fragment outputs* exists edge from any vertex from fragment f to u.

Definition 5.44 (Fragment Inputs)**.** For any given vertices S and F in any given graph G, $f = \Phi(G, S, F)$, define a function $\mathbf{tau}(G, f) = \bigcup_{v \in \Phi(G,\mathrm{S},\mathrm{F})} \{v' \mid v' \in V_G \wedge v' \notin \Phi(G, \mathrm{S}, \mathrm{F}) \wedge (v', v) \in E_G\}$, which evaluates *fragment inputs*.

For any fragment f, *fragment inputs* is a set of vertices, which are not included in f and for any vertex u from *fragment inputs* exists edge from u to any of vertices from f.

Definition 5.45 (Correct Fragment)**.** For any given vertices S and F in any given graph G and $f = \Phi(G, \mathrm{S}, \mathrm{F})$, predicate $\mathbf{\Lambda}(G, f)$ holds if all of the following conditions are met:

- $\sum_{v \in \zeta(G, \Phi(G,\mathrm{S},\mathrm{F}))} \begin{cases} 1 \text{ if } \exists v' \in \mathbf{C}(G, v) : \mathbf{C}(G_c, v') \neq \emptyset \\ 0 \quad \text{otherwise} \end{cases} \leq 1$;
- $|\tau(G, f)| \leq 1$;
- $\nexists v' \in f : \exists v_1, v_2 \in f \cap \mathbf{C}(G, v') : v_1 \neq v_2$;
- $\nexists v' \in f : \exists v_1, v_2 \in f : (v_1 \neq v_2) \wedge (v' \in \mathbf{C}(G, v_1)) \wedge (v' \in \mathbf{C}(G, v_2))$;

Only *correct fragment* can be transformed to code insertion on language $\mathcal{B}$.

In previous definitions in this subsection, the concept of a fragment and a correct fragment was formulated. A fragment is a subset of CFG C vertices that must be converted into a single code insertion in $\mathcal{B}$ with using FFI. Since code insertion using FFI has only one entry point and only one exit point, the subset of vertices whose operations are included in the part of the code transformed to $\mathcal{B}$ must have no more than one edge to a vertex outside the fragment (excluding edges to *sinks*) and no more than one edge included in the fragment from vertex out of fragment.

Definition 5.46 (Set of Correct Fragments)**.** For any given graph G, define a function $\mathbf{\Gamma}(G) = \{\Phi(G, v_1, v_2) \mid v_1, v_2 \in V_G \wedge \mathbf{\Lambda}(G, \Phi(G, v_1, v_2))\}$. This function evaluates all fragments of graph G, which are correct.

Theorem 5.47. *The upper estimate of size of set of correct fragments is:*

$$\forall G \in \mathbb{G} : \ |\mathbf{\Gamma}(G)| \leq |V_G|^2.$$

Proof. For any graph G, for any vertex $v \in V_G$: $|\mathbf{D}(G, v)| \leq |V_G|$. According to Definition 5.41, fragment is uniquely defined by entry-point and exit-point. The total number of possible combinations of a vertex and its descendant is $\sum_{v \in V_G} |\mathbf{D}(G, v)| \leq \sum_{v \in V_G} |V_G| = |V_G| * |V_G| = |V_G|^2$. □

Definition 5.48 (Control Transfer Point)**.** For any given correct fragment f in any graph G, find $v' \in V_G : \ (v', \mathrm{S}) \in E_G \wedge v' \notin f$, name v' *control transfer point*. According to Definition 5.44, when entry-point of f is not source, exists only one *control transfer point*. For fragment f denote its *control transfer point* - $\mathbf{\Theta}(G, f)$.

Definition 5.49 (Control Return Point)**.** For any given correct fragment $f = \Phi(G, \mathrm{S}, \mathrm{F})$ in any given graph G, denote $\mathbf{\Pi}(G, f) = \{v' \in V_G \mid (\mathrm{F}, v') \in E_G \wedge v' \notin f \wedge |\mathbf{C}(G, v')| = 0\}$, name it *set of control return points*. According to Definition 5.43, for correct fragment, $|\mathbf{\Pi}(G, f)| \leq 1$.

Definition 5.50 (Profit)**.** For any $\gamma \in \mathbb{R}$, any given correct fragment $f = \Phi(G, \mathrm{S}, \mathrm{F})$ in any given graph G of any control-flow graph C, let $C' \in X(C)$, where $\mathbf{Y}_{C'}(v) = \begin{cases} \mathcal{A} & v \notin f \\ \mathcal{B} & v \in f \end{cases}$:

$$k_1 = \begin{cases} 1 & |\mathbf{Q}(G) \cup f| = 0 \\ \gamma & |\mathbf{Q}(G) \cup f| \neq 0 \end{cases}$$

$$\phi = \sum_{v \in f} \mathbf{M}(C', v, \mathcal{A})$$

$$r_1 = \sum_{v \in f} \mathbf{M}(C', v, \mathcal{B})$$

$$r_2 = \mathbf{T}(C', \alpha, \beta, \mathbf{\Theta}(G, f), \mathrm{S})$$

$$\mu = \sum_{v \in \mathbf{\Pi}(G,f)} \mathbf{T}(C', \alpha, \beta, \mathrm{F}, v)$$

$$\mathbf{\Psi}(C, f) = k_1 * (\phi - r_1 - r_2 - \mu)$$

$\mathbf{\Psi}(C', f)$ means what expected gain in laboriousness costs of program execution will be obtained if the code of fragment f is transformed into $\mathcal{B}$.

Definition 5.51 (Nonoptimal Fragments)**.** For any given control-flow graph C, denote $\Xi(C) = \{f \in \mathbf{\Gamma}(G_C) \mid \mathbf{\Psi}(C, f) > 0\}$ - *set of nonoptimal fragments*. For any fragment ϕ from *set of nonoptimal fragments*, if ϕ will be transformed to code insertion on language $\mathcal{B}$, then expected execution time of program will decrease.

Lemma 5.52. *The same part of the source code cannot be converted to language $\mathcal{B}$ simultaneously in two different fragments. Then, to maximize possible reduction of laboriousness, for any given $C \in \mathbb{T}$, need to find such a subset of*

$$\omega \subseteq \Xi(C) \colon (\forall f_1, f_2 \in \omega : \ (f_1 \neq f_2) \implies (f_1 \cap f_2 = \emptyset)) \wedge \left(\left(\sum_{f \in \omega} \mathbf{\Psi}(C', f)\right) \to max\right)$$

5.7 Choosing the optimal subset of fragments to transform with using FFI. Reduction to the Maximal clique problem.

The problem statement given in Lemma 5.52 can be reduced to the *maximal clique problem*. In this section, we will show how to reduce the problem to a weighted maximal clique problem.

Definition 5.53 (Clique Graph)**.** For any given control-flow graph C, denote *clique graph* $\psi(C) = (V_{\psi(C)}, E_{\psi(C)})$, $V_{\psi(C)} = \{\langle \xi, \omega \rangle = (\xi_f, \mathbf{\Psi}(C, f)) |\ f \in \Xi(C)\}$, $(\langle f_1, f_2 \rangle \in E_\psi) \Leftrightarrow (f_1, f_2 \in \Xi(C) \wedge |f_1 \cap f_2| \neq 0)$. *Clique graph* is a graph too.

Definition 5.54 (Clique)**.** For any given control-flow graph C, *clique* is $q \subseteq V_{\psi(C)}$.

Definition 5.55 (Independent Clique)**.** For any given $C \in \mathbb{T}$, clique q is *independent* if $\forall v_1, v_2 \in q : v_1 \neq v_2 \implies \langle v_1, v_2 \rangle \notin E_{\psi(C)}$.

In other words, a clique is independent if there is no edge between any pair of vertices included in the clique.

Definition 5.56 (Independent Clique Weight)**.** For any independent clique q in $\psi(C)$ for any control-flow graph C, denote $\mathbf{\Omega}(C, q) = \sum\limits_{v \in q} \omega_v$. It is total weight of all vertices, included in clique q.

Lemma 5.57. *For any given $C \in \mathbb{T}$, to maximize possible reduction of laboriousness, need to find* independent clique $q : \mathbf{\Omega}(C, q) \longrightarrow$ *max. Denote that clique q* ideal independent clique.

Lemma 5.52 and Lemma 5.57 are an equivalent description of which fragments to select to transformation to language $\mathcal{B}$. In addition, according to Definition 5.50, for any $C \in \mathbb{T}$, maximizing $\mathbf{\Psi}_C$ leads to a decrease the value of the **L**-function (Definition 5.38). That is, the task of maximizing $\mathbf{\Psi}_C$ is equivalent to the task of reducing the expected laboriousness of program execution.

6 The algorithm for constructing the clique graph by CFG (α-algorithm) and maximal clique task solving

In the previous section, we described mathematically how to optimally select the code fragments that should be converted into code insertion on language of foreign function interface. This section will provide an algorithm that optimally allocates code fragments so as to minimize the expected execution time of the source program.

6.1 α-algorithm

This section describes an algorithm that builds clique graph by CFG.

First of all, algorithm implementation is *here*.

Here and further, $C \in \mathbb{T}$ is CFG, $V = V_G, E = E_G$.

Input data: α-algorithm takes as input CFG.

The result of the algorithm: according to Lemma 5.52 and Lemma 5.57, algorithm must return set of *correct fragments*, having no intersecting CFG vertices and having maximum total $\mathbf{\Psi}$.

Preprocessing takes place before the algorithm itself:

- For any $v \in V$, the list of programming objects used for reading or writing in all vertices-descendants of v is calculated. Denote objectsUsedInDescendants - array of hashsets, objectsUsedInDescendants[v] contains IDs (ξ) of all programming objects, used in all descendants of v. The calculation of objectsUsedInDescendants[v] is performed using depth-first search (DFS). DFS is started for each vertex. If DFS is started from vertex v, entering new vertex u, all programming objects from $\mathbb{R}(C, u)$ and $\mathbb{W}(C, u)$ are added to objectsUsedInDescendants[v]. Since G in CFG is a rather sparse graph, the DFS single execution time complexity is $O(|V| + |\mathrm{O}|)$. Time complexity of calculation all objectsUsedInDescendants is $O(|V|(|V| + |\mathrm{O}|))$.
- Calculating fragmentRequirements. fragmentRequirements is array of hashsets, each hashset - vertices, color of which should be equal. For any vertex v, if $|\mathbf{C}(G, v)| > 1$, then to fragmentRequirements added hashset, concluding v and all vertices from $\mathbf{C}(G, v)$. Time complexity of calculation fragmentRequirements is $O(|V|)$.
- Calculating verticesInLoops. verticesInLoops is hashset of all vertices, which are members of loop. To calculate verticesInLoops, for each $v \in V$ starts depth-first search. If DFS returns to vertex v in some way, then v puts to verticesInLoops. Since G in CFG is a rather sparse graph, time complexity of calculating verticesInLoops is $O(|V|^2)$.
- Calculating graphReversed for G. graphReversed $= (V, E')$ is graph, where $(a, b) \in E' \Leftrightarrow (b, a) \in E$. Since G in CFG is a rather sparse graph, time complexity of calculating graphReversed is $O(|V|)$.

Functions definition:

findFragments:

The algorithm is based on a DFS. Name that recurcive function *findFragments*($v \in V$). Denote numberPrevObjectUsage - array, which maps programming object ID and number vertices on *findFragments* path to current vetrex v, in which **R** or **W** attended this programming object ID. Each time *findFragments* enters a vertex v, the number of

uses of each programming object from $\mathbf{R}(C, v)$ and $\mathbf{W}(C, v)$ increases by 1, and when exiting, it decreases by 1. Depending on the traversal order, the values in numberPrevObjectUsage may differ, but regardless of the traversal order, if numberPrevObjectUsage$[x] = 0 \implies$ programming object x was never used before vertex v. Entering v, *findFragments* iterates through its descendants w, and checks if $\mathbf{\Phi}(G, v, w)$ correct with using *checkSubFragment*. And after that, it runs recursively from all $\mathbf{C}(G, v)$.

checkSubFragment:

checkSubFragment checks if $f = \mathbf{\Phi}(C, \mathrm{S}, \mathrm{F})$ correct. Firstly, *checkSubFragment* calculates currentFragment - hashset of vertices, included in f. After, *checkSubFragment* is interrupted prematurely if:

- currentFragment contains ρ. According to Lemma 5.18, $\mathbf{Y}(C, \rho_c) = \mathcal{A}$;
- currentFragment contains sink. According to Lemma 5.19, $(|\mathbf{C}(G, v)| = 0) \implies (\mathbf{Y}(C, v) = \mathcal{A})$;
- If it is FALSE, that for all $x \in$ fragmentRequirements, currentFragment contains all vertices from x, or contains no one of them. This check ensures that the all vertices in x, that must be the same color, belong same fragment, or do not belong any fragments. Since requirements are imposed, only on vertices a kind $(a, b \in x : \langle a, b\rangle \in E) \vee (a, b \in x : \exists r \in x : \langle r, a\rangle \in E \wedge \langle r, b\rangle \in E)$, that they can belong only to one fragment, and not to different fragments;
- $\left(\sum_{v \in \zeta(G,f)} \begin{cases} 1 \text{ if } \exists x \in \mathbf{C}(G, v) : \\ |\mathbf{C}(G, x)| \neq 0 \\ 0 \quad \text{otherwise} \end{cases}\right) \geq 2) \vee$ $(|\mathbf{tau}(G, f)| \geq 2)$. According to Definition 5.45, that means F is not correct fragment. To check ζ condition, *checkSubFragment* iterates by $v \in f$ and checks if v has vertex in $\mathbf{C}(G, v)$, which is not sink, and counts the number of such vertices v. Similarly, to check **tau** condition, it iterates by $v \in f$ and counts the number of vertices v for which $|\mathbf{C}(\text{graphReversed}, v)| \neq 0$. Fulfilling this, as well as all the above conditions, ensures that f is correct fragment.
- $\nexists v \in$ currentFragment $: v \in$ prohibitedToTransform. Also, the algorithm provides an opportunity to prohibit the inclusion of certain vertices in fragments;
- $\mathbf{\Psi}(C, f)$ is less or equals 0. The calculation of profit is described in *getFragmentProfit*.

If *checkSubFragment* have not interrupted, it adds fragment f to subFragments array - list of fragments, which will participate in the construction of the cliqueGraph.

getFragmentProfit:

getFragmentProfit returns expected profit of $f = \mathbf{\Phi}(C, \mathrm{S}, \mathrm{F})$. Firstly, it checks if $\exists v \in f : v \in$ verticesInLoops. After that, it counts summarised weight of imported and exported programming objects (with using getFragmentImportObjectsWeight and getFragmentExportObjectsWeight) functions. Finally, it returns profit value (according to Definition 5.50).

getFragmentImportObjectsWeight:

For fragment $f = \mathbf{\Phi}(C, \mathrm{S}, \mathrm{F})$, method calculates all programming objects, occuring in **R** and **W** in all vertices in f (denote fragmentObjects). After that, it counts and returns summarised ω of programming objects in intersection fragmentObjects and programming objects, for which numberPrevObjectUsage is greater 0. This intersection contains only those programming objects that need to be imported into the fragment (according to Definition 5.34).

getFragmentExportObjectsWeight:

For fragment $f = \mathbf{\Phi}(C, \mathrm{S}, \mathrm{F})$, method calculates all programming objects, occuring in **R** and **W** in all vertices in f (denote fragmentObjects). After that, it counts and returns summarised ω of programming objects in intersection fragmentObject and programming objects, for which objectsUsedInDescendants ≥ 1. This intersection contains only those programming objects that need to be exported from the fragment (according to Definition 5.35).

buildCliqueGraph:

buildCliqueGraph builds clique graph by subFragments array. It builds graph $\psi = (V_{CG}, E_{CG})$, $\langle a, b\rangle \in E_{CG}$, if $\exists x : x \in \text{subFragments}_a \wedge x \in \text{subFragments}_b$.

The algorithm:

The algorithm starts with a call *findFragments*(ρ). It is used to calculate subFragments ($\Xi(c)$). After that it calls *buildCliqueGraph*, which builds clique graph for subFragments.

Time complexity:

Prerprocessing time complexity is $O(|V|^2 + |V| * |\mathrm{O}|)$.

findSubFragments time complexity is $O(|V|^3 + |V| * |\mathrm{O}|)$.

buildCliqueGraph time complexity is $O(|\text{subFragments}|^2 * |\mathrm{O}|)$. According to **theorem 5.36**, $|\text{subFragments}| \leq |V|^2$, so buildCliqueGraph time complexity is $O(|V|^4 * |\mathrm{O}|)$. In practice, when considering a large number of CFG examples, $|\text{subFragments}| = O(|V|)$.

So, all α-algorithm time complexity is $O(|V|^4 * |\mathrm{O}|)$.

6.2 Approaches to finding the maximal clique problem

The maximal clique problem is NP-complete task. The search for the maximal clique by a complete search of all options for clique graph with n vertices has time complexity $O(2^n * n^2)$. This is too much computational complexity. In addition, in the original formulation, clique does not have WE (we can assume that for each clique q $\mathrm{WE}_q = 1$). So greedy algorithms

will be used to search for independent clique with as much WE as possible. Greedy algorithms are searching for clique $q : \text{WE}_q \to \max$, but it is not guaranteed that they will find *ideal independent clique*.

Existing algorithms, solving maximal clique problem, which find the exact solution

Unsuccessful attempts were made to modify the following exact algorithms to solve the weighted case of the maximal clique problem:

- Algorithm of Frank Harary and Ian C. Ross (doi:10.2307/2785673).
- Robson algorithm and modification of Robson algorithm. It also has exponential time complexity $O(2^{0.282n}n^2)$, but in practice it shows itself to be much better than a full search.
- Bron and Kerbosch algorithm. It has modification for weighted clique task, but it increases its time complexity to $O(3^{n/3}n)$.
- P. R. J. Ostergard algorithm. In weighted modification, its time complexity is $O(2^n)$, but this algorithm can easily be parallelized, unlike all of the above.

The considered algorithms are applicable in practice only for graphs containing no more than 50-70 vertices, which is a rather serious limitation.

Possible approaches for greedy problem solving:

- *Random walk through the graph:*
  A depth-first search is started from random vertex, at each even iteration (iterations are numbered from 0), the neighbors of the current vertex are added to *skiplist*, and the current vertex is added to the click, and the algorithm switches to a random neighbor of the vertex, which is not included in *skiplist*. On an odd iteration, there is simply a transition to a random neighbor. For a graph with n vertices, n^2 such DFS launches are performed, the one clique with the biggest weight is selected from all.
- *search random cliques:*
  Creates hashset *currennt_clique*. For graph G, while it's possible, select random vertex $v \in V_G$: $(v \notin \textit{current_clique}) \wedge (\nexists x \in \textit{current_clique} : (x, v) \in E_G)$ and add it to *current_clique*. This process repeats for $|V|^2$ times, after that selects clique with maximum WE.

The implemented algorithm takes a cliqueSolver object, which solves maximal clique problem. The implemented algorithm provides several algorithms for solving the maximal clique problem.

7 Proof of the correctness of the α-algorithm

In this section, several statements will be presented, the proof of which justifies the correctness of the α-algorithm.

7.1 The theorem that all returned by α-algorithm fragments are correct and do not intersect

Theorem 7.1. *For any $C \in \mathbb{T}$, algorithm returns set of correct code fragments with no intersections.*

Proof.

- *No vertex should be included in more than one fragment for transformation to language of foreign function interface.*

  According to **definition 5.46**, α-algorithm returns q - *independent clique* of $\psi(C)$. As clique q is independent, $\nexists x, y \in q : (x, y) \in E_{\psi(C)}$. According to **definition 5.44**, that means $\nexists a, b : ((a \neq b) \wedge (f_1, f_2 \in \Xi(C)) \wedge (\xi_{f_1} = a \wedge \xi_{f_2} = b) \wedge (|f_1 \cap f_2| \neq 0))$.

- *Each fragment, which is needed to be transformed to language of foreign function interface, must conclude only one control transfer point.*

  According to **Definition 5.37**, for any fragment that does not conclude ρ_c, it has no more than one *control transfer point*. According to **Definition 5.34**, fragments that conclude ρ_c are not correct. According to **definition 5.19**, ρ_c is a *source*, so $\nexists v \in V_{G_C} : v \neq \rho_c \wedge \rho_c \in \mathbf{D}(G_C, v)$. Thus, each $f \in \Xi(C)$ has only one control transfer point.

- Each fragment, which is needed to be transformed to Rust, must conclude only one control return point.

  According to **definition 5.38**, every correct fragment has no more than one *control return point*. According to **definition 5.18**, every sink has color $\mathcal{A}$, so no one correct fragment contain sink. That means, that for $f = \Phi(G, \mathrm{S}, \mathrm{F})$ for specified S and F, F is not sink. That means, that $|\mathbf{C}(G, \mathrm{F})| \geq 1$. So, $(1 \leq |\mathbf{C}(G, F)| \leq 1) \implies |\mathbf{C}(G, F)| = 1 \implies$ every correct fragment has only one *control return point*.

□

7.2 The theorem on the optimality of fragment selection by α-algorithm

Theorem 7.2. *Using the exact algorithm for solving the* maximal clique problem, *the α-algorithm makes the optimal selection of fragments (according to **definition 5.46**).*

Proof. According to **subsection 5.7**, solving of *maximal clique problem* equals to solving *selecting disjoint fragments*

with the maximum amount of $\mathbf{\Psi}$.

According to **definition 5.30**, every fragment uniquely defined by S and F ($\mathbf{\Phi}_G(\mathrm{S}, \mathrm{F})$).

findFragments function, as it was described, iterates through all such pairs $(\mathrm{S}, \mathrm{F}) : \mathrm{F} \in \mathbf{D}_G(S)$ and filters only *correct* fragments. That means, that *findFragments* finds all correct fragments for building *clique graph*.

α-algorithm returned $w \subseteq \Xi(C)$. Suppose that exists $\omega \subseteq \Xi(C) : \left(\left(\sum_{f \in \omega} \mathbf{\Psi}(C, f)\right) > \left(\sum_{f \in w} \mathbf{\Psi}(C, f)\right)\right) \wedge (\nexists a, b \in \omega : |a \cap b| \neq 0)$. That means, that in $\mathrm{CG}(c)$ exists *independent clique*, which WE is more than WE of maximal independent clique. But that means, that algorithm to solve *maximal clique problem* is not exact, but this contradicts the original assumption. $\perp \implies$ on the contrary, it is shown that with the optimal solution of the *maximal clique problem*, the selection of fragments is optimal.

□

8 α-algorithm hyperparameters

As was mentioned in **subsection 5.4**, for transporting functions were used hyperparameters α, β, γ. This section is devoted to observations on how to select a hyperparameter.

8.1 α-hyperparameter

The hyperparameter α is responsible for the constant part of the cost of transporting data. It includes a constant part of the laboriousness of transporting data to a piece of code converted to the FFI language, and transporting data from a piece of code.

8.2 β-hyperparameter

The hyperparameter β is the coefficient of proportionality of the laboriousness of transporting data to and from a code fragment, depending on the total weight of imported and exported programming objects (according to **definition 5.26**).

8.3 γ-hyperparameter

The hyperparameter γ is the coefficient of regularization for cycles in control flow graph. γ is used only in H-function (**definition 5.27**).

Possible values of γ:

- $\gamma > 1$ does not make sense;
- A value of 1 corresponds to the absence of regularization;
- $\gamma \in (0; 1)$ respond to using regularization. The higher the value of γ, the more the algorithm encourages the conversion of the entire cycle into control flow graph in its entirety, if the profit from one of its iterations is positive. Note that the parameter γ should be the larger the more iterations of the loop in control flow graph are expected on average.

The values of the hyperparameters α, β, γ should be selected experimentally.

9 Results

We took 100 programs in EO and executed them, calculated their evaluation time. Then, we applied our method and executed them again. The difference in performance was noticeable and positive.

EO[1] is an object-oriented programming language based on φ-calculus introduced by Bugayenko [2] and later formalized by Kudasov and Kim [7]. EO is a "more" pure language than Ruby, meaning that it has more objects and less things that are not objects. For example, `to_i` method of `String` class in Ruby is not an object. It is a function implemented in C. EO also has atoms, which essentially are C or Assembly blocks of code, but the percentage of them in the entire standard library is much smaller than of similar entities in Ruby.would be great to show exact numbers here

The file-reading program discussed above may be written in EO as such:

```
[] > file
  [] > eof /bool
  [] > next /string
file > f
memory 0 > a
goto
  [g]
    seq > @
      at.
        QQ.txt.sscanf
          "%d"
          f.next
        0
      if.
        (t.mod 3).eq 0
        a.write
          a.plus
            t.mul t
        g.backward
      if.
        f.eof
        g.forward a
        g.backward
QQ.io.stdout
  QQ.txt.sprintf
    "a = %d"
    a
```

[1] https://www.eolang.org

Our algorithm replaces EO objects with Rust functions. This code may be optimized to the following:

```
goto
  [g]
    rust
      """
      Here goes the code for Rust function:
      pub fn f(&mut uni: Universe, v: u32);
      """
```

Here, the code of Rust function body may look like this:

```
let t = uni.da("|$\Phi$|.f.next.|$\Delta$|").
    to_utf8()?;
if t % 3 == 0 {
  let mut a = uni.da("|$\Phi$|.a.|$\Delta$|").
      to_i64()?;
  a = a + t;
  let write = u.copy("|$\Phi$|.a.write");
  let a0 = u.add();
  uni.bind(write, a0, "|$\alpha$|0");
  uni.put(a0, Hex::from_i64(a));
  uni.da(format!("|$v$|{write}"));
} else {
  uni.da(format!("|$v$|{v}.|$\alpha$|0.backward
      "));
}
let eof = uni.da("|$\Phi$|.f.eof.|$\Delta$|").
    to_bool()?;
if eof {
  let f = u.copy("|$v$|{v}.|$\alpha$|0.forward")
      ;
  uni.bind(f, 0, "|$\alpha$|0/|$\Phi$|.a");
  uni.da(format!("|$v$|{f}"));
} else {
  uni.da(format!("|$v$|{v}.|$\alpha$|0.backward
      "));
}
```

Rust code blocks encapsulated inside `rust` object communicate with other EO objects through Surging Object DiGraph (SODG) notation.

10 Discussion

To be written

11 Conclusion

To be written

12 Conclusion

In this article, the formulation of the algorithmization problem was given, the algorithm for solving this problem was described, and the proof of correctness of algorithm was given. This algorithm is implemented in console tool for optimization code in EO language. Unit tests have been written for this tool, as well as the correctness of the transformations specified by this console tool has been tested on 17 source codes. There are several examples of using the algorithm to optimize code in the performance_test directory of the project repository. There is also a script for measuring the execution time of programs before and after optimization.

References

[1] Zoran Budimlić, Ken Kennedy, and Jeff Piper. 1999. The Cost of Being Object-Oriented: A Preliminary Study. *Scientific Programming* 7, 2 (1999), 87–95. https://doi.org/10.1155/1999/464598

[2] Yegor Bugayenko. 2021. EOLANG and φ-calculus. arXiv:2111.13384 [cs.PL]

[3] Craig Chambers and David Ungar. 1991. Making Pure Object-Oriented Languages Practical. In *ACM SIGPLAN Notices*, Vol. 26. 1–15. https://doi.org/10.1145/118014.117955

[4] Julian Dolby and Andrew Chien. 2000. An Automatic Object Inlining Optimization and Its Evaluation. In *Proceedings of the Conference on Programming Language Design and Implementation (PLDI)*. 345–357. https://doi.org/10.1145/349299.349344

[5] Karel Driesen and Urs Hölzle. 1996. The Direct Cost of Virtual Function Calls in C++. In *Proceedings of the 11th Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*. 306–323. https://doi.org/10.1145/236337.236369

[6] Bruno Dufour, Karel Driesen, Laurie Hendren, and Clark Verbrugge. 2003. Dynamic Metrics for Java. In *Proceedings of the 18th Conference on Object-Oriented Programing, Systems, Languages, and Applications (OOPSLA)*. 149–168. https://doi.org/10.1145/949305.949320

[7] Nikolai Kudasov and Violetta Kim. 2022. φ-calculus: a purely object-oriented calculus of decorated objects. arXiv:2204.07454 [cs.PL]

[8] David Ungar and Randall B. Smith. 1987. Self: The Power of Simplicity. In *ACM SIGPLAN Notices*, Vol. 22. 227–242. https://doi.org/10.1145/38807.38828

[9] David West. 2004. *Object Thinking*. Pearson Education.